

12. HASH-TABLE DATA STRUCTURES

Hash-table principles

Searching

Insertion

Deletion

Hash-table design

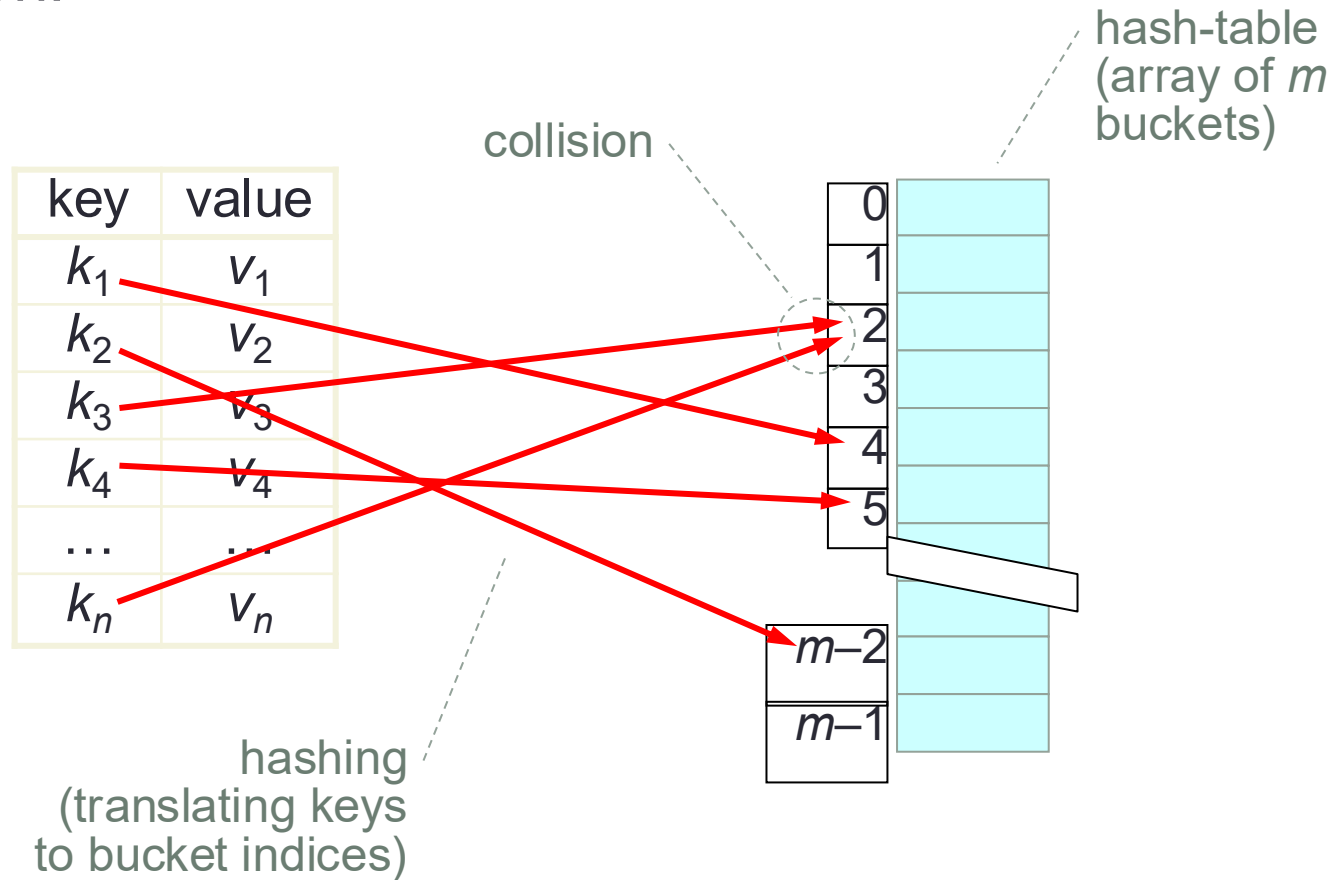
Implementations of sets and maps using hash-tables

Hash-table principles (1)

- If a map's keys are small integers, we can represent the map by a key-indexed array. Search, insertion, and deletion are then $O(1)$.
- Surprisingly, we can approach this performance with keys of *any* type!
- Idea: Translate each key to a small integer, and then use that integer to index an array. This is called **hashing**.
- A **hash-table** is an array of m **buckets**, together with a **hash function** $hash(k)$ that translates each key k to a bucket index (in range $0 \dots m-1$).

Hash-table principles (2)

- Illustration:



Hash-table principles (3)

- Each key k has a **home bucket** in the hash-table, namely the bucket with index $hash(k)$.
- To **insert** a new entry with key k into the hash-table, assign that entry to k 's home bucket.
 - If k 's home bucket is already occupied, we have a collision. This must be resolved somehow.
- To **search** for an entry with key k in the hash-table, look in k 's home bucket.
- To **delete** an entry with key k from the hash-table, look in k 's home bucket.

Hash-table principles (4)

- The hash function must be **consistent**:
 $k_1 = k_2$ implies that $hash(k_1) = hash(k_2)$.
- In general, the hash function is many-to-one.
- Therefore different keys may share the same home bucket:
 $k_1 \neq k_2$ but $hash(k_1) = hash(k_2)$.
If this actually happens, it is called a **collision**.
- Always prefer a hash function that is likely to make collisions infrequent.

Example: a hash function for words

- Suppose that the keys are English words*.
- A possible hash function:
$$m = 26$$
$$\text{hash}(k) = (\text{initial letter of } k) - \text{'A'}$$
- All words with initial letter 'A' share bucket 0; ...; all words with initial letter 'Z' share bucket 25.
- For *illustrative* purposes, this is a convenient choice.
- For *practical* purposes, this is a poor choice: collisions are likely to be freq

*Assume for now all start with a capital letter, or first letter is capitalized before being added to hashtable

Hashing in Java (1)

- hash method in class `Object`:

```
public int hashCode ();  
// Translate this object to an integer, such that  
// x.equals(y) implies that  
// x.hashCode() == y.hashCode().
```

- Note that `hashCode()` is consistent.
- We can use `hashCode()` to implement a hash function for a hash-table with m buckets:

```
static int hash(Object k) {  
    return Math.abs(k.hashCode()) % m;  
}
```

`Math.abs` returns a nonnegative integer.

Modulo- m arithmetic then gives an integer in the range $0 \dots m-1$.

Hashing in Java (2)

- Each subclass of `Object` should override `hashCode`.
- Examples:

Class of k	Result of $k.hashCode()$
<code>String</code>	weighted sum of characters of k
<code>Integer</code>	integer value of k
<code>Date</code>	(high 32 bits of k) exclusive-or (low 32 bits of k) Note: a date k is represented by a long integer, the number of milliseconds since 1970-01-01.

Closed buckets vs open buckets

- In an **open-bucket hash-table**:

- each bucket may be occupied by at most one entry
- whenever there is a collision, displace the new entry to an unoccupied bucket.

not covered here:
see Watt & Brown
and Prosser
additional slides

如果該位置被占了
(collision)
就要 往別的
bucket 探測
(probe)

- In a **closed-bucket hash-table (CBHT)**:

- each bucket may be occupied by several entries
- The destination bucket for any entry is fixed

Closed-bucket hash-tables (1)

- Simplest implementation of a CBHT:
each bucket is an SLL. SLL (Singly Linked List)
- In the following illustrations, the keys are names of chemical elements. For illustrative purposes, assume:

$$m = 26$$

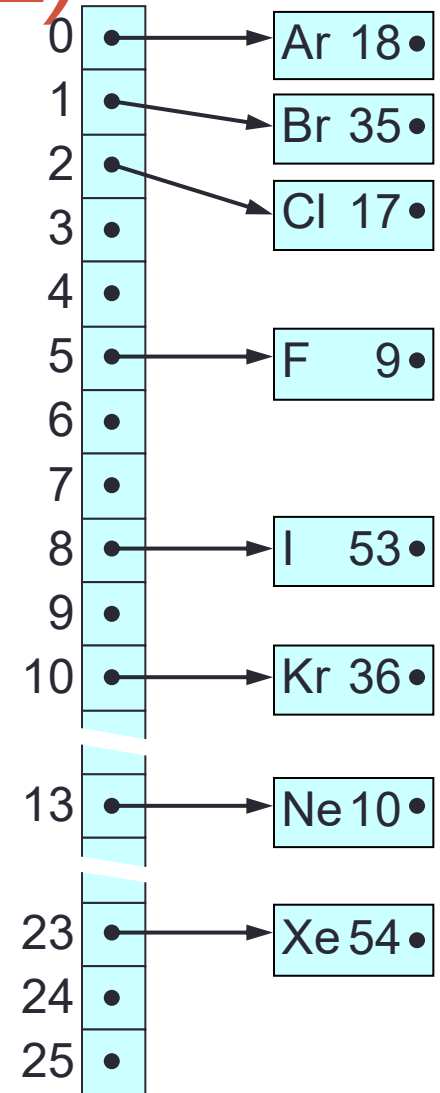
$$\text{hash}(k) = (\text{initial letter of } k) - \text{'A'}$$

Closed-bucket hash-tables (2)

- Illustration (with no collisions):

<u>element</u>	number
F	9
Ne	10
Cl	17
Ar	18
Br	35
Kr	36
I	53
Xe	54

is represented by

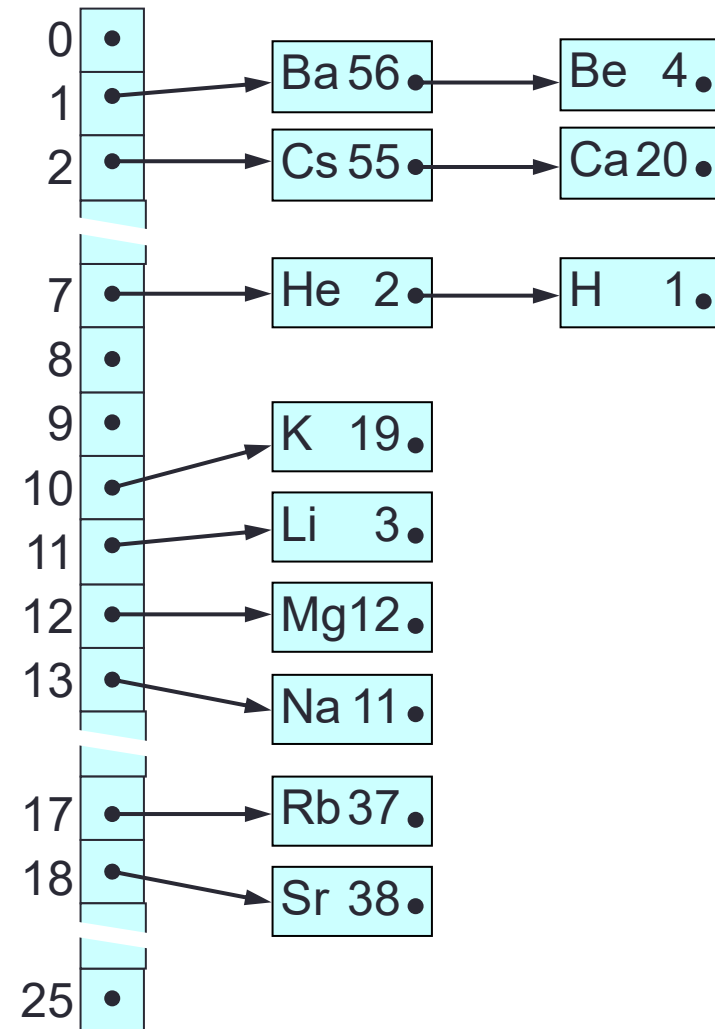


Closed-bucket hash-tables (3)

- Illustration (with collisions):

<u>element</u>	number
H	1
He	2
Li	3
Be	4
Na	11
Mg	12
K	19
Ca	20
Rb	37
Sr	38
Cs	55
Ba	56

is
represented
by



Closed-bucket hash-tables (4)

- Java class implementing CBHTs:

```
public class CBHT<K,V> {  
    private Node<K,V>[] buckets;  
    public CBHT (int m) {  
        buckets = (new Node<K,V>[]) new Node<?,?>[m];  
    }  
  
    private int hash (K key) {  
        return Math.abs(key.hashCode())  
            % buckets.length;  
    }  
}
```

Closed-bucket hash-tables (5)

- Java class (*continued*):

```

////////// Inner class //////////
private static class Node (K,V) {
    // Each Node object is a node of a CBHT.
    private K key, V value;
    private Node<K,V> next;
    private Node (K key, V val,
                  Node<K,V> next) {
        ...
    }
}
}

```

See Moodle for rest of code

CBHT search (1)

- **CBHT search algorithm:**

To find which if any node of a CBHT contains an entry whose key is equal to *target-key*:

1. Set b to $\text{hash}(\text{target-key})$.
2. Find which if any node of the SLL of bucket b contains an entry whose key is equal to *target-key*, and terminate with that node as answer.

Step 2 is SLL linear search.

CBHT insertion and deletion

- **CBHT insertion algorithm:**

To delete the entry (if any) whose key is equal to key from a CBHT:

1. Set b to $hash(key)$.
2. Search bucket for any entry whose key is equal to key
If found update entry
Otherwise add new entry to front of list
3. Terminate.

- **CBHT deletion algorithm:**

To delete the entry (if any) whose key is equal to key from a CBHT:

1. Set b to $hash(key)$.
2. Delete the entry (if any) whose key is equal to key from the SLL of bucket b .
3. Terminate.

CBHTs: analysis

- Analysis of CBHT search, insertion, and deletion algorithms (counting comparisons):

Let the number of entries be n .

- In the **best case**, no bucket contains more than (say) 2 entries:

Max. no. of comparisons = 2

Best-case time complexity is $O(1)$.

- In the **worst case**, one bucket contains all n entries:

Max. no. of comparisons = n

Worst-case time complexity is $O(n)$.

CBHTs: design

- CBHT design consists of:
 - choosing the number of buckets m
 - choosing the hash function *hash*.
- Design aims:
 - collisions should be infrequent
 - entries should be distributed evenly among the buckets, such that few buckets contain more than (say) 2 entries.

CBHTs: choosing the number of buckets

- The **load factor** of a hash-table is the average number of entries per bucket, n/m .
- If n is (roughly) predictable, choose m such that the load factor is likely to stay in range 0.5...0.75.
 - A load factor < 0.5 wastes space.
 - A load factor > 0.75 tends to result in some buckets having many entries.
- Where possible, choose m to be a **prime number**.
 - Typically the hash function performs modulo- m arithmetic. If m is prime, the entries are more likely to be distributed evenly over the buckets, regardless of any pattern in the keys.

質數 (又稱素數)

CBHTs: choosing the hash function

- The hash function should be efficient (performing few arithmetic operations).
- The hash function should distribute the entries evenly among the buckets, regardless of any patterns in the keys.
- Possible compromise:
 - Speed up the hash function by using only part of the key.
 - But beware of any patterns in that part of the key.

Example: hash-table for words (1)

- Suppose that a hash-table will contain about 1,000 common English words.
- Known patterns in the keys:
 - Letters vary in frequency
(A, E, I, N, S, T are common;
Q, X, Z are uncommon).
 - Word lengths vary in frequency
(lengths of 4–8 are common;
other lengths are less common).

Example: hash-table for words (2)

- $hash(k)$ can depend on any of k 's letters and/or length.
- Consider $m = 20$, $hash(k) = \text{length of } k - 1$.
 - Far too few buckets. Load factor = $1000/20 = 50$.
 - Very uneven distribution.
- Consider $m = 26$, $hash(k) = \text{initial letter of } k - \text{'A'}$.
 - Far too few buckets. Load factor = $1000/26 \approx 38$.
 - Very uneven distribution.

Example: hash-table for words (3)

- Consider $m = 520$, $\text{hash}(k) = 26 \times (\text{length of } k - 1) + (1^{\text{st}} \text{ letter of } k - \text{'A'})$.
 - Too few buckets. Load factor = $1000/520 \approx 1.9$.
 - Very uneven distribution.

Words of length 4 are far more common than words of length 2, so buckets 78–103 will be far more densely populated than buckets 26–51.

Words with 1st letter 'A' are far more common than words with 1st letter 'Z', so buckets 0, 26, 52, ... will be far more densely populated than buckets 25, 51, 77, ...

And so on.

Example: hash-table for words (4)

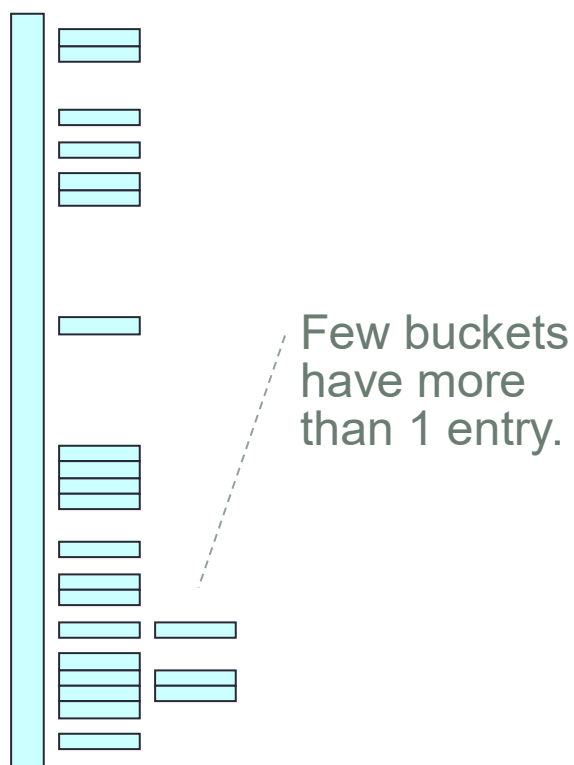
- Consider $m = 1499$, $hash(k) =$ (weighted sum of letters of k) modulo m
 - i.e., $(c_1 \times 1^{st} \text{ letter} + c_2 \times 2^{nd} \text{ letter} + \dots)$ modulo m
 - + Good number of buckets. Load factor = $1000/1499 \approx 0.67$.
 - + Reasonably even distribution.

Hash-tables in practice (1)

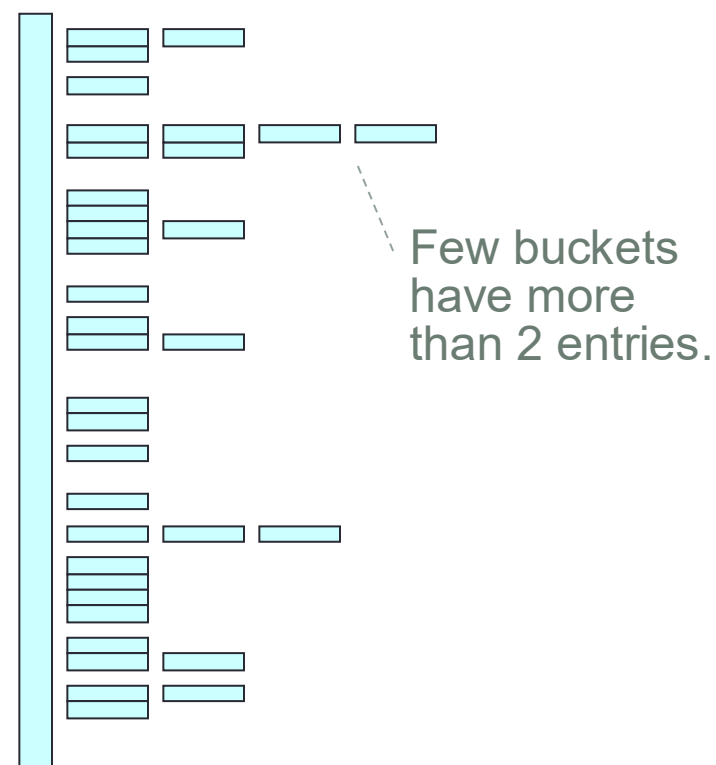
- The following trials illustrate the performance of a CBHT with m buckets.
- The hash function was chosen to distribute keys uniformly among the m buckets (i.e., the probability that a key is mapped to any particular bucket was $1/m$).
- In each trial, the CBHT was loaded with a set of n randomly-generated keys.

Hash-tables in practice (2)

- Trials with $m = 47$:



CBHT
 $n = 23$ (load factor ≈ 0.5)



CBHT
 $n = 36$ (load factor ≈ 0.75)

Example: student records (1)

- Consider a hypothetical university's student records. About 1,500 new students register each year, and most students stay for 4 years.
- Each student has a unique student-number of the form *yydddd*
 - *yy* are the last two digits of the year of first registration
 - *dddd* is a serial number.
- Suppose that the student records will be held in a hash-table.
- $hash(k)$ can depend on any or all of the digits of the student-number k .

Example: student records (2)

- Consider $m = 100$, $hash(k) = \text{first 2 digits of } k$.
 - Far too few buckets. Load factor ≈ 60 .
 - Very uneven distribution. E.g., in academic year 2022–23, most student-numbers start with 19, 20, 21, or 22.
- Consider $m = 10,000$, $hash(k) = \text{last 4 digits of } k$.
 - + Good number of buckets. Load factor ≈ 0.6 .
 - Uneven distribution. Most student-numbers end with 0000...1500.
- Consider $m = 9,973$, $hash(k) = k \text{ modulo } m$.
 - + Good number of buckets. Load factor ≈ 0.6 .
 - + Even distribution (since whole key is used and m is prime).

每年約 1500 個新生，學生大多待 4 年，
所以系統裡同時大約有：

$n \approx 1500$
 $\times 4 = 6000$

所以如果：

• $m = 10000$, load factor $\approx 6000 / 10000 = 0.6$

• $m = 9973$, load factor $\approx 6000 / 9973 \approx 0.6$

Implementation of maps using CBHTs – if both **maps** implemented as CBHTs with sizes n and n' and hash function

- Summary of algorithms:

Operation	Algorithm	Time complexity	
		best	worst
get	CBHT search	$O(1)$	$O(n)$
remove	CBHT deletion	$O(1)$	$O(n)$
put	CBHT insertion	$O(1)$	$O(n)$
putAll	merge on corresponding buckets of both CBHTs	$O(m)$	$O(n'n)$
equals	equality test on corresponding buckets of both CBHTs	$O(m)$	$O(n'n)$

Maps have sizes n and n' where ($n=n'$ for equals) and linked lists are sorted (only have access to head of list)

Implementation of maps using CBHTs – assuming if we don't know second map's implementation (just iterator)

- Summary of algorithms:

Operation	Algorithm	Time complexity	
		best	worst
get	CBHT search	$O(1)$	$O(n)$
remove	CBHT deletion	$O(1)$	$O(n)$
put	CBHT insertion	$O(1)$	$O(n)$
putAll	CBHT insertion for each entry of second map	$O(n')$	$O(n'n)$
equals	CBHT search for each entry of second map	$O(n')$	$O(n'n)$

Maps have sizes n and n' where ($n=n'$ for equals) and linked list of (first) CBHT are unsorted

Implementation of sets using CBHTs (if both sets implemented using CBHTs with same size and hash function)

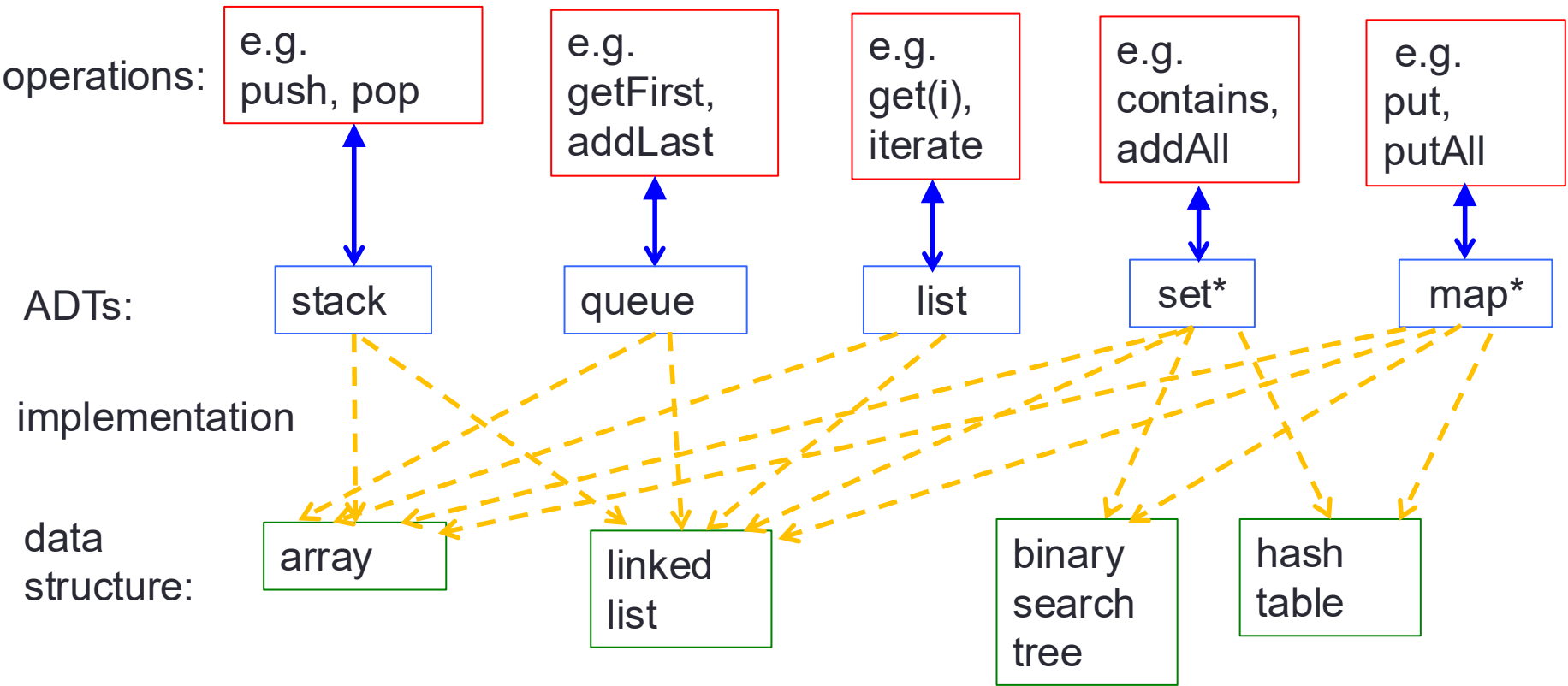
- Summary of algorithms:

Operation	Algorithm	Time complexity	
		best	worst
<code>contains</code>	CBHT search	$O(1)$	$O(n)$
<code>add</code>	CBHT insertion	$O(1)$	$O(n)$
<code>remove</code>	CBHT deletion	$O(1)$	$O(n)$
<code>equals</code>	equality test on corresponding buckets of both CBHTs	$O(m)$	$O(n'n)$
<code>containsAll</code>	subsumption test on corresponding buckets of both CBHTs	$O(m)$	$O(n'n)$
<code>addAll</code>	merge on corresponding buckets of both CBHTs	$O(m)$	$O(n'n)$
<code>removeAll</code>	variant of merge on corresponding buckets of both CBHTs	$O(m)$	$O(n'n)$
<code>retainAll</code>	variant of merge on corresponding buckets of both CBHTs	$O(m)$	$O(n'n)$

Open bucket hash tables OBHTs

- We don't do these, but there is an example demo for one implementation (linear probing) due to Daniel Laing here:
<https://yongdanielliang.github.io/animation/web/LinearProbing.html>
Link is given on Moodle
- Some extra slides in which OBHTs are discussed (Prosser) are also available on Moodle.

Final diagram – (didn't make it into the video!)



if 大範圍整數 用 Array : index = key 這件事會讓 array 在「大範圍整數」時直接爆掉 (記憶體或不可行)。

Hash Table 沒這問題因為只存「有用的 key」

- 空間 $\approx O(n)$ ，不是 $O(\text{range})$
- 適合 sparse data

* + special boolean/value array implementations when entries/keys are small integers – equivalent to HTs when keys have this form